Milestone 2 Design brief and systems plan

I will first construct the <theory-of-the-program>, then generate <program text> only after the theory is explicit.

OPERATIONAL_DIAGNOSTIC

Substantives_Demolished Replace vague uses of clarity, system and feasibility with the actions and records below.

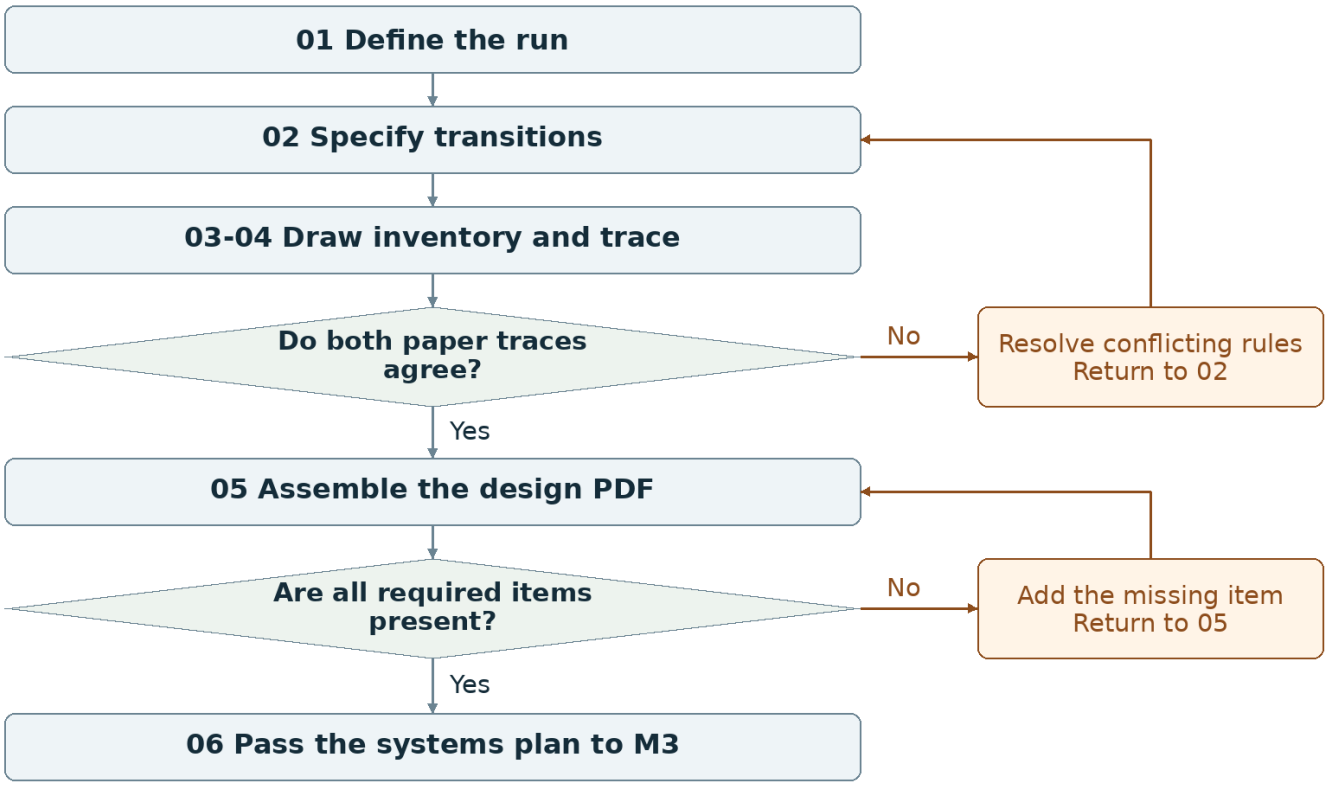
Occult_States_Removed Replace both partners understand the design with two written traces of the same inputs and a comparison of their outputs.

The_Language_Game Write the state, accepted input, condition, data change and visible response for every planned transition.

CONTROL_SURFACE_DRAFT

Write enough rules for both partners to predict a run, then submit the brief, program state flowchart, asset list and two mockups.

Start with Open the selected M1 concept, rationale, sketch, sources and unresolved choices. Reopen M1 if no concept has been adopted.



Use the numbered procedure; repeat checks only where the repair affects their result.

Required package Submit one group PDF of at least two pages; keep the design brief itself to one page.

Assign maker _____ and checker _____. Swap roles at the next task. Leave checks pending until performed.

Milestone 2 Write the rules

1 <Initial Interpretation>

Write the chosen sequence: <player> [starts] <run> and [uses] <control>; <input handler> [checks] <condition> and [updates] <domain value>; <outcome handler> [records] <outcome> when the written test succeeds. State the goal, unsuccessful condition and ending; use only the scoring or competition your concept actually needs.

2 <Theory Skeleton>

Record	What to write
<Entities>	List each domain value and its purpose. Give initial values, allowed values and the event that changes each value.
[Operations]	For each control, record the permitted action, input validation, data change and displayed result.
<States>	Name at least three states for the required program diagram. Record visible information and accepted or blocked inputs in each.
<Conditions>	Write a testable condition on every transition. Set priority if two ending conditions can hold at once.
<Invariants>	Write rules that must remain true within stated scopes. Example to adopt or replace: after an ending, ordinary play input cannot change the completed result.
<Constraints>	List platform, required screens/states, input method, asset limit and recipient launch requirement. Separate these delivery constraints from runtime invariants.

Use conditions to decide whether an action or transition is allowed. State the scope of each invariant and how to check it. Keep submission requirements separate from runtime rules.

3 <Assumption Ledger>

Status	Action
<safe>	Enter an unverified premise only with its basis and limited consequence if wrong. Keep adopted choices in the entry records and requirements in constraints.
<uncertain>	Mark timing, randomness, difficulty, file loading and device layout untested. Name a prototype check for each.
<requires-user-decision>	Resolve disputed goal, ending, controls and simultaneous-outcome priority before implementing those rules.

Record {claim, status, owner, check, consequence if wrong}. Give each unresolved implementation question an M3 test. Rewrite contradictory core rules now.

Record the next open item Claim _____ Status _____
Owner _____ Evidence/check _____ Next action _____

Write <entity> for a person, object, value or state; [operation] for an action; and {record} for related fields. Record planned, observed, failed or not tested beside each claim. Do not mark a check passed until a person performs it and records the result.

Milestone 2 Run the procedure

4 <Operational Description>

Read each action aloud before doing it. The maker performs the action; the checker records the named result. Use the return instruction when a check fails.

Step	Perform and save	Check and return
01 Fix the run rule	Both partners complete the purpose sentence and mark the required goal, unsuccessful condition and ending. List what the player sees before deciding.	Remove rules that require information the interface does not show, or add that information to the mockup.
02 Specify transitions	Create one row per event: {current state, input/event, condition, data change, next state, visible result}. Add blocked-input handling and reset values if replay is included.	Trace every row from a defined start. Repair unreachable outcomes, undefined values and conflicting conditions.
03 Draw and inventory	Draw the program state flowchart with at least three states. Draw both title and main-interaction mockups. List each asset's use, creator/source, permission or license, status and owner.	Match control labels to transition rows. Mark placeholders and remove optional assets that exceed the work allowance.
04 Run paper checks	Give both partners the same starting values and input sequence. Each writes the states, data and outputs independently. Compare one complete run and one blocked input.	Resolve differences in step 02. Update the diagram and mockups after a rule changes. Do not settle a contradiction by choosing different test inputs.
05 Assemble the PDF	Write the one-page brief from the agreed rules. Add flowchart, asset list, both mockups, screenshot/capture, progress note and AI statement on further pages.	Open the exported PDF. Confirm total length is at least two pages, the brief is one page and each figure can be read.
06 Hand off	Give M3 the brief, transition table, state diagram, mockups, asset list, scoped invariants and test list. Record actual installed versions when available.	Both partners locate the same start values, ending condition and first prototype check.

5 <Failure Description>

Observed problem	Required response
Different predicted outputs	Revise the relevant condition in step 02; repeat the same paper trace.
Workflow map submitted as state diagram	Create the program's screen/state diagram in step 03. The process map in this manual does not satisfy that deliverable.
Missing title or main mockup	Draw the absent interface and annotate its controls; include both in step 05.

Use the map A no branch returns to the named task. Repair the recorded problem and repeat that check; do not mark the stage passed because the planned time window ended.

Milestone 2 Test changes and produce the work

6 <Change Test>

Ask: "What would change if the requirements changed?" The maker writes the expected result first. The checker performs or traces the change and records the result. Swap roles for the second scenario.

Modification	Run this check
Change the limiting rule	Replace a timer with a fixed number of decisions; if no limit exists, propose a decision limit. Revise state data, conditions and instructions on paper. Trace one ending with the replacement.
Add another outcome	Add one proposed ending. Write its entry condition and priority against existing endings. Trace a case where two conditions coincide; specify exactly which result is shown.

Save {version or design record, starting condition, input/change, expected result, observed result, pass/fail/not tested, next action}. Use paper traces in M1-M2; report executed checks only when they occurred.

7 <Implementation Plan>

Assign the transition table and first paper trace to one partner, the mockups and asset inventory to the other, then swap review. Outline initialization, input handling, rule updates, drawing, outcome checks and reset if used. Schedule peer availability early; collect peer evidence in M4.

8 <Program Text>

Produce the design documents. No game implementation is required in M2. Use short pseudocode only to settle a specific rule, after writing that rule in the transition table; keep it labeled proposed.

9 <Theory-Code Mapping>

Element	Record the connection
<types>	Propose a data representation for each entity and valid range.
<functions>	Assign each operation and condition to one planned responsibility.
<classes>	Add a class only for a named recurring responsibility; otherwise mark not used.
<tests>	Link each condition or invariant to a paper trace and future run check.
<comments>	Record why outcome priority, reset or input restrictions were chosen.
<configuration>	List adjustable limits, sizes and timings separately from fixed rules.

Use actual file and function names when code exists. Label planned counterparts proposed; write not applicable when a structure is absent.

Carry forward the course rules

Keep the project connected to WarGames. Build in Processing using p5.js mode; accept interactive input; include at least three distinct screens or states; provide a start and reachable outcome; credit external material; organize and comment the code.

Record actual AI use at every milestone, including use of this manual if adopted. Use AI for bounded brainstorming, explanation, small examples you study and adapt, specific debugging or review. Author the project as a group; do not request an entire generated project. Identify assisted code and your changes in comments.

Milestone 2 Submit and hand off

10 <Residual Human Theory>

Save the reasons for the chosen mechanic, outcome priority and asset cuts. Each partner traces the same proposed rule change through the table and mockup. Write any disagreement as a specific rule to resolve before its code is written.

Check the required submission

- One group PDF, at least two pages, containing a one-page design brief.
- A program state flowchart with at least three states and labeled transitions.
- An asset list.
- A title-screen mockup and a main-interaction-screen mockup.
- At least one screenshot/mockup capture, short progress note, credits as needed and AI Use Statement.

Suggested order: page 1 brief; page 2 program flowchart and asset list; page 3 or later mockups and records. Keep the brief's one-page requirement distinct from total PDF length.

Completion check The checker finds every required document. Both partners' traces give the same next states and visible outputs for the agreed input sequence; each planned outcome has an explicit entry condition.

Progress note Name the current version; state what was completed and checked, what is missing, and the next action with its owner. Add the screenshot/capture supporting the current status.

Complete the AI Use Statement

Field	Enter the actual record
Tool and dates	Name the tool, or record no AI used only if accurate. Enter actual dates of use.
Request and affected work	Summarize the request. Name the text, image, planning record or code file affected.
Kept changed rejected	Record the output retained, edits or rewrites made, and material discarded.
Group confirmation	Recommended check: each partner traces actual submitted code; for planning-only work, trace the proposed interaction. Identify image sources, retained edits and displayed information. Then complete the required group confirmation covering code and images; leave it pending until both partners can confirm it.

If this AI-produced manual is adopted, record ChatGPT, October 21 2026, milestone planning and operational rewrite, and your actual kept/changed/rejected choices. Do not copy a completed student confirmation from this manual.

Handoff Pass the agreed rules and unresolved technical questions to M3. Mark the workflow map in this manual as a work aid, not the required program state diagram.

Record: maker _____ checker _____ version _____
Checks still pending _____ submitted file(s) _____ upload status _____

Requirements source: supplied WarGames project brief and milestone instructions. Treat roles, file names, internal checks and relative-week allocations as recommended working procedures. Use corrected deadlines from the course; none are asserted here.